

Campus Art Fest ("Rendezvous") â€™ Project Overview

Comprehensive reference for the **ISRA life Festival 2026** web app. Written from the actual codebase in this repository. Anything that could not be confirmed from code is marked `TODO: confirm`.

1. Product Overview

The **Campus Art Fest** web app (marketed as "**Rendezvous â€™ ISRA life Festival 2026**") is the digital hub for an annual campus arts festival. It brings together four audiences:

Audience	What they can do
General public / visitors	Watch the hero gallery, view live team standings, browse teams / students / programmes, see finished results, download result posters, download gallery photos, and read about the festival.
Students	Log in with name + password to edit their own profile (photo, category, programmes) and see their participation progress and earned points.
Judges	Log in, submit results per programme (1st / 2nd / 3rd with points), and later re-edit submitted results through a re-verified, audited flow.
Admin	Manage programmes, students, teams, spotlight gallery; run lot draws; generate and publish cumulative result posters; produce printable committee sheets; preview all results.

The mission statement used on the site: "**Track, Celebrate, Remember.**"

- Live site: `isra-rendezvous.vercel.app` (deployed on Vercel)
- Source repo: `github.com/itsmewhizz/Artfest-Web-app`
- Backend: Supabase (Postgres + Auth + Storage)

2. Tech Stack

Layer	Technology	Notes
Framework	React 19 (via Vite 8)	SPA, client-side routing
Language	JavaScript (JSX)	No TypeScript
Routing	react-router-dom v7	BrowserRouter ; see routes below
Styling	Tailwind CSS 3 + custom <code>index.css</code>	Color tokens exposed as CSS variables mapped to Tailwind colors

Layer	Technology	Notes
Icons	lucide-react	
Backend	Supabase (<code>@supabase/supabase-js</code>)	Postgres DB, Row Level Security, Auth (email/password for admin & judges), Storage bucket <code>photos</code>
Poster generation	html2canvas + file-saver	Client-side PNG rendering/download
Charts / progress	react-circular-progressbar	Student participation ring
Fonts	Playfair Display, Poppins, Inter (Google Fonts)	Loaded in <code>src/index.css</code>
Linting	oxlint	<code>npm run lint</code>
Build	<code>vite build</code>	PowerShell blocks npm scripts; run via <code>cmd /c "npm run build"</code>

Frontend architecture

- No global state library â€” plain `useState` / `useEffect` per page.
- Data access is centralized in `src/supabase/queries.js` (reusable read helpers).
- Supabase clients live in `src/supabase/client.js` (3 clients â€” see Security section).
- Global toast notifications via a React context (`src/components/Toast.jsx` , `ToastProvider` wraps the app in `src/main.jsx`).

Routes (from `src/App.jsx`)

Path	Page	Guard
<code>/</code>	Home	â€”
<code>/teams</code>	Teams	â€”
<code>/teams/:id</code>	TeamDetail	â€”
<code>/students</code>	Students	â€”
<code>/students/:id</code>	StudentProfile	â€”
<code>/programmes</code>	Programmes	â€”
<code>/programmes/:id</code>	ProgrammeResult	â€”
<code>/student/login</code>	StudentLogin	â€”
<code>/student/dashboard</code>	StudentDashboard	session check
<code>/admin/login</code>	AdminLogin	â€”
<code>/admin</code>	AdminDashboard	<code>ProtectedRoute</code> (Supabase session)

Path	Page	Guard
/admin/programmes	AdminProgrammes	ProtectedRoute
/admin/teams	AdminTeams	ProtectedRoute
/admin/spotlight	AdminSpotlight	ProtectedRoute
/admin/students	AdminStudents	ProtectedRoute
/admin/print	AdminPrint	ProtectedRoute
/admin/results	AdminResults	ProtectedRoute
/admin/result-poster	AdminResultPoster	ProtectedRoute
/admin/lots	AdminLots	ProtectedRoute
/judges/login	JudgesLogin	â€”
/judges/results	JudgesResults	JudgesRoute (role must be judge)

3. Features by Section

3.1 Home (/)

- **Transparent top nav** with the ISRA Festival wordmark.
- **Full-screen hero** that cross-fades featured spotlight images every 5 s (falls back to an animated gradient when empty), with an aurora animation layer, sparkles, and a starfield background.
- **Team Standings** â€” animated stacked "HP bars" per team, colored by category, with a live count-up of total points. Each bar opens a **TeamBreakdown modal** (1st/2nd/3rd counts, category breakdown, optional "View Full Details").
- **Download Total Result** â€” lists *published* cumulative posters (stored in localStorage under `result_posters`) with a per-poster download button.
- **Gallery** â€” horizontal, snap-scrollable grid of spotlight images, each with a download button.
- **About the Fest** + footer. The bottom nav "Home" icon scrolls to the hero; hamburger menu's "About" smooth-scrolls to the About section.

3.2 Teams (/teams) & Team Detail (/teams/:id)

- Teams ranked by total points (computed from latest result per programme via `getTeamCategoryPoints`).
- Each team bar shows the same animated standings; clicking opens the breakdown modal; "View Full Details" goes to `/teams/:id` .
- **TeamDetail** has two tabs:
 - **Students** â€” list of the team's students (with category filter chips), each showing their points earned and programme badges; click through to `/students/:id` .

- **Places** â€” a 1st / 2nd / 3rd toggle listing the programmes the team won, with result numbers and points; click through to `/programmes/:id`.

3.3 Students (`/students`) & Student Profile (`/students/:id`)

- **Students list** â€” search by name, filter by team and category (via `FilterDropdown`). Cards show avatar, name, team and class; click for the profile.
- **StudentProfile** â€” avatar, name, team & class, a **circular progress ring** (`finished` / `total` programmes participated), and a **count-up points earned** number. A hidden category filter (click the points) breaks down programme list by category. Two tabs:
 - **Completed** â€” finished programmes with placement points (click to open the poster modal).
 - **Pending** â€” registered programmes not yet conducted (click through to the programme page).
 - Each completed programme row has a **Download Poster** button (uses `ResultPoster`).

3.4 Programmes (`/programmes`) & Programme Result (`/programmes/:id`)

- **Programmes list** â€” search + category + type filters (On-stage / Off-stage), shows `#resultNo`, category, type, and a **Finished / Unfinished** status.
- **ProgrammeResult** â€” programme card with Finished/Pending pill. When finished, shows the podium (1st/2nd/3rd with medals, avatars, points) and a **Download Poster** button that opens the themed `ResultPoster` modal (Classic / Vibrant / Minimal themes) rendered with `html2canvas` and saved via file-saver.

3.5 Student panel (`/student/login` , `/student/dashboard`)

- **StudentLogin** â€” name + password against the `student_credentials` table (exact `ilike` name match first, then fuzzy substring). Distinct error messages for: not found, no credentials set, wrong password, already logged in elsewhere.
- On success the dashboard opens in a **new tab** (see User Flows).
- **StudentDashboard** â€” edit own name, category, profile photo (uploaded to `photos` storage), and select participating programmes (checkboxes). Saves via `updateStudentProfile`. Sessions expire after 8 hours (`SESSION_EXPIRY_MS`) and are enforced in `localStorage` + `students` DB fields.

3.6 Judges panel (`/judges/login` , `/judges/results`)

- **JudgesLogin** â€” email/password sign-in via a dedicated `judgeClient` (separate storage key so it never collides with the main client).
- **JudgesResults**:
 - Programme list with category filter; every row has an **Edit** button (even when locked).
 - **Submitted Results** section lists locked results with expandable 1st/2nd/3rd rows (points + grade badges).

- **Edit flow (locked results):** Edit â€˜ "Are you really a Judge?" prompt â€˜ **re-verify** (judge name + password + server-issued CAPTCHA) â€˜ edit form â€˜ **Save & Lock**.
- The save is the `judge_reverify_edit` SECURITY DEFINER RPC, which re-checks judge role, name/email match, password hash, and the single-use 5-minute CAPTCHA **on the server**, then upserts the result, re-locks it, and appends an audit row to `result_edit_log`.

3.7 Admin panel (`/admin` + 8 sub-screens)

- **AdminDashboard** â€˜ grid of tiles linking to all sub-screens, plus logout.
- **AdminProgrammes** â€˜ add programmes (name, category, type, optional result number), toggle Finished, edit (modal with name/category/type/resultNo), and a participants modal showing which students are enrolled per programme. "Print" button jumps to `/admin/print`. Result numbers are managed through *unlocked placeholder result rows*.
- **AdminLots** â€˜ lot-draw tool. Two modes: **Topic** (reveal order numbers 1â€˜N) and **Code Letter** (Aâ€˜Nth). Cards are 3D-flip cards; shuffle randomizes the pool; max 60 cards / 26 letters. Pure client-side (no DB writes).
- **AdminTeams** â€˜ team scoreboard sorted by total points with 1st/2nd/3rd counts, expandable `TeamBreakdown` modal with category filter.
- **AdminStudents** â€˜ add/edit students (name, category, team, photo upload, programme checkboxes split into category-specific and "General Programmes" groups), category filter chips.
- **AdminSpotlight** â€˜ upload images to `photos` storage, set an optional title-cased caption, toggle **featured** (hero banner) vs gallery-only, and delete.
- **AdminResults** â€˜ **read-only** preview of all results (latest per programme), searchable by programme name or result number, expandable rows with points + grade. Explicitly judge-write only.
- **AdminResultPoster** â€˜ generate a **cumulative team poster** through N programmes (must be a positive multiple of 5, â‰¥ total programmes). Renders a gradient poster via `html2canvas`, stores it in `localStorage` (`result_posters`), and lets the admin **publish/unpublish** it (published posters appear in the Home downloads area, synced across tabs via the `storage` event).
- **AdminPrint** â€˜ printable committee sheets. Tabs for **Programmes** (participant list) and **Results** (top-3 with points/grades). Supports multi-select (max 4 sheets) or single preview, an optional **committee stamp** image (persisted in `localStorage` key `printStampImage`), and A4 portrait print CSS (`@media print` hides admin chrome).

4. User Flows

4.1 Admin login â€˜ panel (new tab)

1. Admin enters email + password on `/admin/login`.
2. The handler opens a blank tab **synchronously** (`window.open('', '_blank')`) so browsers don't block it as a popup.

3. `supabase.auth.signInWithPassword` runs; on success the popup is pointed to `/admin`, and the original tab navigates back to `/`.
4. On failure the popup is closed and an inline error is shown.
5. If the popup was blocked (`popup === null`), the app falls back to same-tab navigation to `/admin`.

The exact same new-tab pattern is used for the **Judges login** (`â†’ /judges/results`) and **Student login** (`â†’ /student/dashboard`).

4.2 Student login â†’ dashboard (session-managed)

1. Student enters full name + password.
2. `getStudentByCredentials` looks up the student (exact name first, then fuzzy), verifies the password against `student_credentials`, and rejects if a session is already active elsewhere.
3. A unique token is generated and stored both in `localStorage` (`student_id`, `student_session_id`, `student_session_expires_id`) and in the `students` row (`sessionActive`, `sessionExpiresAt`, `sessionToken`).
4. Dashboard opens in the new tab; session auto-expires after **8 hours** and is enforced on dashboard load.
5. Logout clears the session locally and in the DB.

4.3 Judge edits a (locked) result â€” re-verified + audited

1. Judge taps **Edit** on any programme row.
2. Modal: "Are you really a Judge?" â†’ Yes.
3. **Re-verify screen:** judge enters name + password; the client requests a server CAPTCHA via `judge_create_captcha()` (6-char, single use, 5-min expiry). The client checks the CAPTCHA, then also calls `verifyJudgeClient.auth.signInWithPassword` (a throwaway client that never touches the judge's active session) to confirm the entered name/password belongs to a `judge`.
4. **Edit form:** pick 1st/2nd/3rd students (filtered to the programme's category unless General) and points; a live grade badge updates as points are typed.
5. **Save & Lock** calls the `judge_reverify_edit` RPC which *independently* verifies: caller is a judge â†’ name matches the JWT email â†’ password matches the hash in `auth.users` (`crypt`) â†’ CAPTCHA is valid/unused/unexpired â†’ then upserts the result, sets `locked = true`, and writes old/new placements to `result_edit_log`.
6. Result list refreshes; the result remains locked. Direct RLS still blocks any UPDATE/DELETE on locked rows â€” this RPC is the only write path.

4.4 Admin generates + publishes a cumulative poster

1. In `/admin/result-poster`, enter a programme count (multiple of 5).
2. Server data (teams, students, programmes, latest results) is loaded; the cumulative points per team are summed and sorted.

3. The preview is rendered to a PNG with `html2canvas` and stored in `localStorage[ 'result_posters' ]` (offline â€œ survives refresh, not shared across devices).
4. The admin toggles the poster **published**.
5. Home listens for the `storage` event and lists published posters under "Download Total result" for visitors.

4.5 Visitor downloads a result poster

- From a programme page or student profile, "Download Poster" opens the `ResultPoster` modal with three themes (Classic / Vibrant / Minimal). The poster shows the programme name, `#resultNo`, category, and 1st/2nd/3rd rows with avatars, points and grade. "Download Poster" renders the element with `html2canvas` and saves a PNG named `<programme>_poster.png`.

4.6 Admin prints committee sheets

1. `/admin/print` â†’ choose **Programmes** or **Results** tab, filter by category.
2. Tap an item to preview one sheet, or enter **Select** mode to pick up to 4 items and print them together.
3. Optionally upload a **committee stamp** image (stored in `localStorage`).
4. Preview shows A4 sheets (category, event name, number, stamp box, SI.NO / NAME / TEAM [+ POINT / GRADE for results]).
5. **Print** triggers `window.print()`; print CSS hides admin chrome and lays out one sheet per page.

5. Design System

5.1 Color tokens (`src/index.css` :root + `tailwind.config.js`)

Colors are RGB triplets stored as CSS variables and exposed to Tailwind via `rgb(var(--token) / <alpha-value>)`, so any class like `bg-primary` / `text-mutedText` works with opacity modifiers.

Token	RGB	Hex	Role
<code>--main-background</code>	40 114 161	<code>#2872A1</code>	Page background (ocean blue)
<code>--primary</code>	31 90 128	<code>#1F5A80</code>	Deep navy â€œ buttons, active states
<code>--secondary</code>	92 147 170	<code>#5C93AA</code>	Slate teal â€œ borders, secondary UI
<code>--accent</code>	232 132 92	<code>#E8845C</code>	Sunset coral â€œ gold, points, highlights
<code>--main-text</code>	234 244 250	<code>#EAF4FA</code>	Light text on dark background
<code>--muted-text</code>	169 199 214	<code>#A9C7D6</code>	Muted / secondary text
<code>--success</code>	76 187 130	<code>#4CBB82</code>	Success, LOCKED, Finished, toasts
<code>--card</code>	22 64 92	<code>#16405C</code>	Deep navy card surface
<code>--ocean-tint</code>	132 186 225	<code>#84BAE1</code>	Light tint â€œ public section cards

Hardcoded extra colors used in UI: `#0F2A3D` (dark navy text on light surfaces), `#CBDDE9` (light dropdown/modal surfaces), `#7FC3EA` (accent icons), `#5FB3DF` (scrollbar).

5.2 Category color palette

Defined in `TeamBar.jsx`, `TeamBreakdown.jsx`, `TeamDetail.jsx` and `AdminPrint.jsx` (each has a local copy).

Category	Light	Dark
Minor	<code>#55EFC4</code>	<code>#00B894</code>
HS	<code>#FF7675</code>	<code>#D63031</code>
Premier	<code>#74B9FF</code>	<code>#0984E3</code>
Sub Junior	<code>#A29BFE</code>	<code>#6C5CE7</code>
Junior	<code>#FDCB6E</code>	<code>#D68910</code>
General	<code>#D1D5DB</code>	<code>#9CA3AF</code>

Used for category filter chips, stacked standing bars, and badge gradients (`linear-gradient(135deg, light, dark)`).

5.3 Typography

- **Playfair Display** (`font-display`) â€” headings, hero, poster titles.
- **Poppins** (`font-poppins`) â€” UI headings, numbers.
- **Inter** (`font-inter`) â€” body text.
- Body default is Inter (`body { font-family: 'Inter', sans-serif }`).

5.4 Motion & backgrounds

- **Starfield** â€” twinkling fixed stars (`Starfield.jsx`).
- **Aurora blobs** â€” three drifting blurred radial gradients.
- **Sparkles** (hero) and **embers** (standings card) â€” rising particles.
- **Bar grow** â€” team bars animate via `IntersectionObserver` + `requestAnimationFrame` with easing and per-bar stagger (`STAGGER_MS = 150` , `GROW_MS = 2250`).
- **Count-up** â€” `useCountUp` hook eases numbers (used for points).
- All decorative animations are disabled under `prefers-reduced-motion` .

5.5 Reusable components

- `TeamBar` â€” animated standing bar (exports `CATEGORY_COLORS`).
- `TeamBreakdown` â€” expandable team stats modal.
- `StudentAvatar` â€” image with initial-letter fallback.
- `FilterDropdown` â€” click-outside-close dropdown with icons.

- `ResultPoster` â€” themed downloadable poster modal.
- `Toast` / `ToastProvider` / `useToast` â€” success/error toasts (3 s auto-dismiss).
- `BottomNav` â€” floating pill nav (Home / Teams / Students / Programmes); **hidden on panel routes** (`/admin`, `/student/*`, `/judges/*`) by matching the first path segment, so the public `/students` section keeps the nav.
- `HamburgerMenu` â€” Home-only top-right menu (Pages, Logins, About).

6. Data Model (Supabase)

Tables

`students` â€” `id` (uuid), `name`, `class` (category), `team` (team id or name), `photoURL`, `programmeIds` (jsonb array of programme ids), `createdAt`, plus session columns `sessionActive` (bool), `sessionExpiresAt` (timestamp), `sessionToken` (text).

`programmes` â€” `id`, `name`, `category`, `isFinished` (bool), plus `programmeType` (On-stage / Off-stage) and a legacy `type` / `programme_type` fallback read by `getProgrammeType`.

`teams` â€” `id`, `name`, `color`, `totalPoints` (maintained by DB trigger, see below).

`results` â€” `id`, `name`, `programmeId`, `first` / `second` / `third` (each a jsonb object { `studentId`, `name`, `points`, `grade`, `photoURL` }), `updatedAt`, `locked` (bool), `resultNo` (int). A programme may have multiple historical rows; the **latest by `updatedAt`** is the effective result.

`spotlight` â€” `id`, `imageUrl`, `caption`, `isFeatured` (bool), `uploadedAt`.

`student_credentials` â€” `student_id`, `password` (plain text â€” see Known Issues).

`add_missing_credentials.sql` inserts `password123` for every student missing a record.

`judge_captcha_challenges` (from `judge_reverify_flow.sql`) â€” `id` (uuid), `judge_id`, `captcha`, `expires_at` (default now + 5 min), `used` (bool). RLS-enabled; only the SECURITY DEFINER functions touch it.

`result_edit_log` (from `judge_reverify_flow.sql`) â€” `id`, `result_id`, `programme_id`, `programme_name`, `judge_email`, `edited_at`, `old_first` / `old_second` / `old_third` and `new_first` / `new_second` / `new_third` (jsonb). Audit trail for re-verified edits; admins can SELECT.

Storage

- Bucket `photos` â€” paths `spotlight/<ts>_<file>` (AdminSpotlight) and `students/<studentId>_<ts>.<ext>` (StudentDashboard / AdminStudents). Public URLs used directly in rows.

Database functions & triggers

- `sync_team_totals()` â€” SECURITY DEFINER trigger function; on any change to `results` it recomputes `teams.totalPoints` from the latest result per programme joined to students â†’ teams.
- **Trigger** `trigger_sync_team_totals` on `results` (AFTER INSERT/UPDATE/DELETE, statement-level).

- `public.is_admin()` / `public.is_judge()` â€” read the JWT `app_metadata.role`.
- `public.judge_create_captcha()` (RPC, SECURITY DEFINER) â€” issues a 6-char captcha (charset ABCDEFGHJKLMNPQRSTUVWXYZ23456789) for a logged-in judge.
- `public.judge_reverify_edit(...)` (RPC, SECURITY DEFINER) â€” the only write path for locked results; re-verifies role/name/password/captcha, upserts, re-locks, audits.

LocalStorage keys

- `student_id`, `student_session<id>`, `student_session_expires<id>` â€” student session.
- `result_posters` â€” generated cumulative posters ({ [count]: { programmeCount, generatedAt, imageUrl, entries, published } }).
- `printStampImage` â€” committee stamp data-URL for the print screen.

Categories / constants

- `PROGRAMME_CATEGORIES` = ['General', 'Minor', 'HS', 'Premier', 'Sub Junior', 'Junior']
- `STUDENT_CATEGORIES` = ['Minor', 'HS', 'Premier', 'Sub Junior', 'Junior'] (no General)
- `PROGRAMME_TYPES` = ['On-stage', 'Off-stage']
- `SESSION_EXPIRY_MS` = 8 * 60 * 60 * 1000
- Grading: 10 â†’ A+, 8â€”9 â†’ A, 6â€”7 â†’ B, 4â€”5 â†’ C, else - (`calcGrade`).

7. Security Rules (RLS + auth)

RLS is enabled on: `students`, `programmes`, `results`, `teams`, `spotlight`, `judge_captcha_challenges`, `result_edit_log`

Read policies

- **Public read** on `students`, `programmes`, `results`, `teams`, `spotlight` (TO public USING (true)).
- **Admin read** of `result_edit_log` (`is_admin()`).

Write policies

- `students` / `programmes` / `teams` / `spotlight` â€” admin only for INSERT/UPDATE/DELETE (`is_admin()`).
- `results` :
 - `judge_insert_results` â€” judges may INSERT **only rows with** `locked = true`.
 - `admin_insert_result_placeholders` â€” admins may INSERT only placeholder rows (no placements, `locked = false`).
 - `judge_update_results` â€” updates only on **unlocked** rows, and only by judges (or admins on empty placeholders).

- There is **no delete policy** for results â€” locked rows are immutable via SQL; the RPC is the sanctioned exception.
- `judge_captcha_challenges` â€” no direct table policies; access is exclusively through the SECURITY DEFINER functions.

Auth model

- **Three Supabase clients** (`src/supabase/client.js`):
 - `supabase` â€” default client, key `sb-<project>-auth-token` ; used for the site and admin login.
 - `judgeClient` â€” storage key `artfest-judge-auth` ; used for judge sign-in and all judge RPCs.
 - `verifyJudgeClient` â€” `persistSession: false` , storage key `artfest-verify-auth` ; throwaway client that re-verifies judge credentials without disturbing the active judge session or colliding with the default client key (which previously caused the "Multiple GoTrueClient instances" warning).
- **Roles** come from `auth.users.raw_app_meta_data.role = 'admin' or 'judge'` (see `judges_rls.sql` , which assigns `judge` to a hard-coded email list and `admin` to everyone else). After role changes, users must log out/in so the JWT picks up the new `app_metadata` .
- **Guards:**
 - `ProtectedRoute` â€” requires any Supabase session (`/admin/*`).
 - `JudgesRoute` â€” requires a session whose `app_metadata.role === 'judge'` (`/judges/results`).
 - Student dashboard is gated by the 8-hour session token (`localStorage` + DB).
- **Server-side re-verification** for editing locked results (judge name = JWT email, password hash via `crypt` , single-use 5-min CAPTCHA) is enforced inside `judge_reverify_edit` .

Security caveats (TODO: confirm / address)

- `student_credentials.password` and `results.*.points /grades` are plain values in the DB; the student credential check compares plain text (`eq('password', password)`). The judges' passwords, by contrast, are verified via `auth.users` password hashes.
 - RLS **does not** restrict *reads* â€” anyone can read results/students (intended for the public site).
 - `result_edit_log` values are stored as JSON without hash â€” logs are append-only by design but not tamper-evident.
 - CAPTCHA is client-rendered text (not an image) â€” it slows humans down, but is not a strong anti-bot measure by itself.
-

8. Known Issues / In Progress

Not yet deployed / requires action

1. `judge_reverify_flow.sql` **has NOT been run in Supabase yet.** The judges' re-verified edit flow (captcha table, audit log, `judge_create_captcha`, `judge_reverify_edit` RPCs) only works once this file is executed in the Supabase SQL Editor. Until then, judge edits on locked results will fail.
2. **Uncommitted changes.** The new-tab logins (Student/Judge/Admin), the judges re-verify flow, the `BottomNav` first-segment fix, and `judge_reverify_flow.sql` are all uncommitted. Commit + push so Vercel redeploys the latest code.
3. Existing users (admins and judges) must log out and back in after role metadata is set so their JWT includes the new role.

By design (worth documenting, not necessarily bugs)

- **Judges' password** is stored by Supabase Auth (hashed). **Students' passwords** live in plain text in `student_credentials` (see Security caveats).
- **Result posters** and the **committee stamp** live in `localStorage` – “device-local, lost on cache clear, and not shared between devices/browsers. `result_posters` syncs across tabs via the `storage` event only on the same device.
- `teams.totalPoints` is a denormalized column maintained by the `sync_team_totals` trigger; other screens (e.g. `getTeamCategoryPoints`) recompute points client-side from results, so totals shown can differ depending on the source.
- **Student team linkage** is by *name* or *id* (`teamNameToId` lookups handle both), which is flexible but means renaming a team can break existing student links.

Known code smells / TODOs

- `calcGrade` and `CATEGORY_COLORS` are duplicated in several files (`ProgrammeResult` -adjacent pages, `JudgesResults`, `AdminPrint`, `TeamBar`, `TeamBreakdown`, `TeamDetail`, `AdminStudents` -related pages, `Students`). A shared constants module would remove drift risk. TODO: consider centralizing.
- `AdminStudents.handleAdd` sets `createdAt` on *update* too (updates `createdAt` when editing a student) – “confirm intended.
- `AdminResults`, `AdminResultPoster` and others load all students/programmes/results into memory and compute client-side – “fine at festival scale, but not scalable.
- `getTeamCategoryPoints` recomputes points from raw results rather than trusting `teams.totalPoints`, so the Home/Teams standings and the DB column can disagree if the trigger was ever disabled (see `disable_trigger.sql`).